Imagine it’s opposites day and you want addition to replace subtraction (and vice versa) in your CoffeeScript program. How can you swap them? By manipulating the AST! First, use the compiler to generate an AST for the expression `2 + 1`:

```
coffee = require 'coffee-script'
nodes = coffee.nodes '2 + 1'
```

The node for this addition expression has an `operator` and `first` and `last` properties representing the left and right sides of the operator, respectively:

```
addition = nodes.expressions[0]
addition.operator
# '+'

addition.first.base.value
# '2'

addition.second.base.value
# '1'
```

When you compile these nodes you get the JavaScript you expect:

```
nodes.compile bare: true
# 'return 2 + 1;'
```

Now, if you change a node, you’ll change the compiled output:

```
addition.operator = '-'
nodes.compile bare: true
# 'return 2 - 1'
```

Great! You know how to manipulate the AST, but what can you *actually do* with that?

Generating code

Imagine you’re working in a team and they don’t write tests. In the following listing you can see a basic implementation of using the AST to generate files that contain tests for class methods. This same technique could be used dynamically to look at code coverage.